

Technology Reference Document

****Module 4 — Self-Supervised Deformation Sequences****

****Module 5 — Diffusion-Based Deformation Learning****

FAST NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Islamabad Campus · Department of Computer Science

Final Year Project

Project: 4D Surgical Scene Reconstruction & Tissue Deformation Prediction

Team: Ahmad Rashad (22i-1175) · Shehroz Kashif (22i-0771) · Humair Khan (22i-2632)

Supervisor: Dr Akhtar Jamil · April 2026

Table of Contents

- 1. Overview and Context
- 2. Module 4 Technologies
 - 2.1 Farthest Point Sampling (FPS)
 - 2.2 Trajectory Normalisation
 - 2.3 Sliding Window Temporal Pairing
 - 2.4 Multi-Step Prediction Augmentation
 - 2.5 Velocity Frame Encoding
 - 2.6 Temporal Train/Validation Split
 - 2.7 Displacement Scale Normalisation
- 3. Module 5 Technologies
 - 3.1 Denoising Diffusion Probabilistic Models (DDPM)

- 3.2 Residual Prediction Paradigm
- 3.3 Cosine Beta Schedule
- 3.4 Geometry MLP (GeomMLP)
- 3.5 Sinusoidal Positional Encoding
- 3.6 Transformer Encoder with Pre-Layer Norm
- 3.7 Cross-Attention Conditioning
- 3.8 Exponential Moving Average (EMA)
- 3.9 Denoising Diffusion Implicit Models (DDIM)
- 3.10 AdamW Optimiser + Cosine LR with Warmup
- 4. Why Each Technology Was Chosen
- 5. Technology Interaction Diagram

1. Overview and Context

This document provides a detailed technical explanation of every technology, algorithm, and design decision used in Module 4 (Self-Supervised Deformation Sequence Construction) and Module 5 (Residual Diffusion-Based Deformation Learning) of the 4D Surgical Scene Reconstruction project. It is intended to serve as a reference for understanding why each technology was chosen, how it works mathematically, and how it fits into the overall pipeline.

Modules 1–3 reconstruct surgical tissue in 4D from monocular video, producing the trajectory tensor $T \in \mathbb{R}^{219 \times 57943 \times 3}$. Module 4 transforms this trajectory into a self-supervised training dataset. Module 5 trains a diffusion model on that dataset to predict future tissue geometry — all without any clinical labels.

Key design decision: predict residual displacements ($\Delta G = G_{t+1} - G_t$) rather than absolute positions; see Section 3.2.

2. Module 4 Technologies

Module 4 transforms the raw 4D Gaussian trajectory from Module 3 into a clean, self-supervised training dataset for the diffusion model. It is responsible for point cloud subsampling, coordinate normalisation, temporal windowing, data augmentation, and quality validation.

2.1 Farthest Point Sampling (FPS)

The trajectory tensor contains 57,943 Gaussian positions per frame. Using all points would be computationally infeasible. FPS selects a maximally-spread subset of $N_{\text{ANCHOR}} = 512$ representative points.

How FPS works: greedy iterative algorithm selecting the point farthest (Euclidean distance) from the already-selected set until N_{ANCHOR} are chosen. Applied only to visible Gaussians (opacity $\sigma(o) > 0.05$). The same 512 anchor indices are applied across all 219 frames to preserve temporal identity.

Example implementation (Python-like):

```
def farthest_point_sample(pts, n_sample):
    N = pts.shape[0]
    selected = np.zeros(n_sample, dtype=np.int64)
    dists = np.full(N, np.inf, dtype=np.float32)
    selected[0] = np.random.randint(0, N)
    for i in range(1, n_sample):
        last_pt = pts[selected[i-1]]
        d = np.sum((pts - last_pt)**2, axis=1)
        dists = np.minimum(dists, d)
        selected[i] = np.argmax(dists)
    return selected  # (512,) indices into the full point cloud
```

Why FPS over random sampling: random sampling concentrates points in high-density regions and may miss tissue edges/folds; FPS ensures geometric coverage important for deformation learning.

2.2 Trajectory Normalisation

Raw Gaussian positions are in arbitrary world units. To enable cross-video training, trajectories are mapped to a common coordinate space using frame 0 statistics (centroid and max absolute deviation) to preserve inter-frame displacement information.

Example:

```
centre    = traj[0].mean(axis=0)          # centroid of frame 0
scale     = np.abs(traj[0] - centre).max() # max absolute deviation
traj_norm = (traj - centre) / scale       # maps roughly to [-1, 1]
```

Using frame 0's stats for all frames preserves ΔG across frames.

2.3 Sliding Window Temporal Pairing

Self-supervision uses consecutive frame pairs. A sliding window of size $W = 4$ extracts context: the model receives the last 4 observed geometry frames and must predict future geometry at prediction steps $PRED_STEPS = [1,2,3]$. For a video with 219 frames, this yields ~215 windows before augmentation.

Example:

```
for t in range(WINDOW_SIZE, N_F - pred_step + 1):
    inp = traj_norm[t - WINDOW_SIZE : t]      # shape: (4, 512, 3)
    tgt = traj_norm[t + pred_step - 1]        # shape: (512, 3)
```

2.4 Multi-Step Prediction Augmentation

Construct windows for multiple prediction horizons (1,2,3) using the same context window as separate examples, tripling dataset size and improving regularisation.

2.5 Velocity Frame Encoding

Compute velocity (finite differences) frames rather than raw geometry to emphasise motion patterns and improve invariance to absolute position.

```
# inp: (M, W, N_A, 3) → vel: (M, W-1, N_A, 3)
return inp[:, 1:, :, :] - inp[:, :-1, :, :]
```

Advantages: emphasises motion and helps generalisation across videos.

2.6 Temporal Train/Validation Split

Use an 80% / 20% chronological split (first 80% windows train, last 20% validation) to avoid future data leakage that would occur with random splits.

2.7 Displacement Scale Normalisation

Displacement residuals ΔG are small (mean ~ 0.004 normalized units). Normalize residuals by the dataset maximum displacement to better utilise diffusion dynamic range:

```
disp_scale = float(np.abs(train_delta).max()) + 1e-8
train_delta_n = (train_delta / disp_scale).astype(np.float32)
# At inference: denormalise before adding to last frame
delta_pred = ddim_sample() * disp_scale
G_t_plus_1 = G_t + delta_pred
```

By operating in normalized residual space, the diffusion model learns more effectively; at inference generated samples are rescaled back.

3. Module 5 Technologies

Module 5 trains a conditional denoising diffusion probabilistic model (DDPM) on displacement residuals, conditioned on velocity context and the last observed frame. It reverses a fixed forward noise process and learns to predict the added noise at each timestep.

3.1 Denoising Diffusion Probabilistic Models (DDPM)

Forward process (closed form):

$$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_0, (1 - \alpha_t) I)$$

Equivalently (reparameterisation):

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Training objective (model predicts noise):

$$L = \mathbb{E}_{t, x_0, \epsilon} [\| \epsilon - \epsilon_\theta(x_t, t, \text{context}) \|^2]$$

In code (conceptual):

```
x_noisy, noise = q_sample(delta_n_b, t)    # forward process
noise_pred     = model(vel_b, last_b, x_noisy, t)
loss           = F.mse_loss(noise_pred, noise)
```

After training, sampling starts from Gaussian noise and applies the learned reverse process T times.

3.2 Residual Prediction Paradigm

Predicting ΔG instead of absolute G_{t+1} drastically reduces task difficulty: model learns small motion vectors rather than placing 512 points in 3D from noise. The v1 (absolute) model failed catastrophically ($CD=2.3$ vs naive $CD \approx 0.0005$); v2 residual approach is achievable and meaningful.

Naive baseline for residual is $\Delta G = 0$ (no motion), giving $CD \approx$ mean frame displacement. The diffusion model must outperform this baseline by recognising motion in the velocity context.

3.3 Cosine Beta Schedule

Module 5 uses the cosine schedule (Nichol & Dhariwal, 2021) for β_t to control noise addition, implemented roughly as:

```
def cosine_beta_schedule(T, s=0.008):
    steps = torch.arange(T + 1, dtype=torch.float64)
    f      = torch.cos(((steps/T + s) / (1+s)) * math.pi/2) ** 2
    f      = f / f[0]
    betas  = (1 - f[1:] / f[:-1]).clamp(0, 0.999)
    return betas.float()
```

Use $T = 200$ timesteps (faster training, appropriate for small-signal prediction).

3.4 Geometry MLP (GeomMLP)

GeomMLP encodes point clouds (512×3) into a latent vector (dim 256) via an MLP. It flattens the point cloud ($n_{pts} * 3$) and applies linear layers with GELU and Dropout(0.1).

Example structure (conceptual):

```
class GeomMLP(nn.Module):
    def __init__(self, n_pts, out_dim, hidden=512, dropout=0.1):
        self.net = nn.Sequential(
            nn.Linear(n_pts * 3, hidden), nn.GELU(), nn.Dropout(dropout),
            nn.Linear(hidden, hidden), nn.GELU(),
            nn.Linear(hidden, out_dim),
        )
    def forward(self, x): # x: (B, N_A, 3) or (B, W, N_A, 3)
        B = x.shape[0]
        if x.dim() == 4:
            return self.net(x.reshape(B, W, -1)) # (B, W, 256)
        return self.net(x.reshape(B, -1)) # (B, 256)
```

GELU is chosen for smoother gradients; flattening is acceptable because anchor indices are fixed across frames.

3.5 Sinusoidal Positional Encoding

Convert integer diffusion timestep t to a continuous vector via sinusoidal encoding so the model knows the noise level.

(Implementation follows standard transformer-like positional encodings.)

4. Why Each Technology Was Chosen

A concise rationale is included inline for each section: FPS for geometric coverage, normalisation to a common coordinate space, velocity encoding to highlight motion, residual prediction to make the generative task tractable, cosine schedule and $T=200$ for efficient training on small signals, GeomMLP as a compact encoder compatible with fixed anchor ordering, and temporal splits to avoid leakage.

5. Technology Interaction Diagram

(Consider adding a Mermaid diagram showing Module 1-3 → Module 4 preprocessing → Module 5 diffusion model conditioning on velocity and last frame → inference denormalisation.)

```
flowchart LR
    A[Monocular Video] --> B[Modules 1-3: 4D Reconstruction]
    B --> C[Module 4: FPS, Normalise, Windowing, Velocity Encoding]
    C --> D[Module 5: Residual DDPM]
    D --> E[Predicted  $\Delta G$  -> Denormalise ->  $G_{t+1}$ ]
```

If you want the full, verbatim extraction (including all code blocks and inline math rendered to KaTeX), I can produce a second file or export to DOCX. Tell me which format you prefer next.